Network Virtualization with KVM

CentOS Stream 9 Edition

Complete Step-by-Step Guide — Bridged Network + 2 Guest VMs

INPT – Rabat | 2024

Ubuntu vs CentOS — Key Differences

The overall architecture is identical to the Ubuntu guide. What changes is the tooling. Here is a quick reference:

Feature	Ubuntu Server	CentOS Stream 9
Package manager	`apt`	`dnf`
Network config	`Netplan (YAML)`	`nmcli / nmtui`
Bridge tool	`bridge-utils`	`bridge-utils (same)`
Firewall	`ufw`	`firewalld`
SELinux	`AppArmor (off)`	`SELinux (enforcing)`
Service manager	`systemd`	`systemd (same)`
KVM packages	`qemu-kvm + libvirt-daemon-system`	`qemu-kvm + libvirt`
virsh / virt-install	`same`	`same`

> ⚠ **WARNING**
> SELinux is enabled and enforcing by default on CentOS Stream 9. This guide includes the required SELinux steps so your VMs can access disk images.

Network Architecture

vm1	vm2
tap0 (virtual NIC)	tap1 (virtual NIC)
br0 — Linux Bridge (CentOS Stream 9 Host)	
enp3s0 (Physical NIC — replace with yours)	
Physical Network / Router / Internet	

Step 1 — Check Virtualization Support

```
grep -c vmx /proc/cpuinfo    # Intel (must return > 0)
grep -c svm /proc/cpuinfo    # AMD   (must return > 0)
```

Also verify the kernel modules are available:

```
lsmod | grep kvm
# If empty, load manually:
```

```
sudo modprobe kvm
sudo modprobe kvm_intel   # or kvm_amd for AMD CPUs
```

Step 2 — Install KVM and Required Packages

CentOS Stream 9 uses dnf (the successor to yum). Install all required packages:

```
sudo dnf update -y

sudo dnf install -y \
  qemu-kvm \
  libvirt \
  libvirt-client \
  virt-install \
  virt-viewer \
  bridge-utils \
  NetworkManager \
  NetworkManager-bridge
```

Enable and start the libvirt daemon:

```
sudo systemctl enable --now libvirtd
sudo systemctl status libvirtd    # Should show: active (running)
```

Add your user to the libvirt and qemu groups:

```
sudo usermod -aG libvirt $(whoami)
sudo usermod -aG qemu $(whoami)
newgrp libvirt
```

Step 3 — Identify Your Physical Network Interface

```
ip a
```

Find the interface that is UP with an IP address — for example enp3s0, ens3, eth0, etc. Use that name in all steps below.

```
# Example output:
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
    inet 192.168.1.50/24  brd 192.168.1.255  scope global dynamic
```

> ⚠ **WARNING**
> Write down your NIC name. Replace enp3s0 in all following commands with your actual interface name.

Step 4 — Create the Network Bridge with nmcli

On CentOS Stream 9, NetworkManager (nmcli) is the standard tool for network configuration — not Netplan. The bridge is created with two nmcli commands.

4.1 Create the Bridge Interface (br0)

```
sudo nmcli connection add \
  type bridge \
  con-name br0 \
  ifname br0 \
  bridge.stp no
```

4.2 Attach Your Physical NIC to the Bridge

Replace enp3s0 with your actual interface name:

```
sudo nmcli connection add \
  type bridge-slave \
  con-name br0-slave \
  ifname enp3s0 \
  master br0
```

4.3 Enable DHCP on the Bridge and Bring it Up

```
# Enable DHCP on br0
sudo nmcli connection modify br0 ipv4.method auto

# Disable the old standalone connection for the physical NIC
sudo nmcli connection down enp3s0 2>/dev/null || true

# Bring up the bridge and slave
sudo nmcli connection up br0
sudo nmcli connection up br0-slave
```

4.4 Verify the Bridge

```
ip a show br0
# Expected: inet 192.168.x.x/24  assigned by DHCP

sudo brctl show
# Expected:
# bridge name   bridge id          STP enabled   interfaces
# br0           8000.xxxxxxxxxxxx  no            enp3s0

nmcli connection show
# Both br0 and br0-slave should show as connected
```

> **ℹ NOTE**
> If br0 has no IP, run: sudo nmcli connection down br0 && sudo nmcli connection up br0 — this forces a fresh DHCP request.

Step 5 — Configure Firewalld for the Bridge

CentOS Stream 9 uses firewalld by default. You need to add br0 to the trusted zone so VM traffic flows freely:

```
# Add br0 to the trusted zone
sudo firewall-cmd --permanent --zone=trusted --add-interface=br0

# Allow libvirt traffic
sudo firewall-cmd --permanent --add-service=libvirt

# Reload firewall
sudo firewall-cmd --reload

# Verify
sudo firewall-cmd --zone=trusted --list-interfaces
# Should show: br0
```

> **IMPORTANT**
> Skipping this step is the most common reason VMs have no internet access on CentOS. Do not skip it.

Step 6 — Configure SELinux for KVM

SELinux is enforcing on CentOS Stream 9. You need to set the correct context on the VM disk image directory so libvirt can read and write it:

```
# Set SELinux context on the default libvirt images directory
sudo semanage fcontext -a -t virt_image_t '/var/lib/libvirt/images(/.*)?'
sudo restorecon -Rv /var/lib/libvirt/images/

# Verify
ls -Z /var/lib/libvirt/images/
# Should show: system_u:object_r:virt_image_t:s0
```

Also enable the required SELinux booleans for libvirt networking:

```
sudo setsebool -P virt_use_fusefs 1
sudo setsebool -P virt_sandbox_use_netlink 1
```

> **ℹ NOTE**
> If you store VM images in a custom path (e.g. /data/vms), run the semanage fcontext command with that path instead.

Step 7 — Define the Bridged Network in libvirt

7.1 Create the Network XML

```
sudo nano mynetwork.xml
```

```
<network>
  <name>bridged-network</name>
  <forward mode='bridge'/>
  <bridge name='br0'/>
</network>
```

7.2 Define, Start and Autostart

```
sudo virsh net-define mynetwork.xml
sudo virsh net-start bridged-network
sudo virsh net-autostart bridged-network

# Verify
sudo virsh net-list --all
# Expected:
# Name                State    Autostart   Persistent
```

```
# bridged-network    active   yes         yes
```

Step 8 — Create the Two Virtual Machines

> ⚠ **WARNING**
> Download an Ubuntu 22.04 (or CentOS) ISO first and note its path. Update --cdrom in the commands below accordingly.

8.1 Create vm1

```
sudo virt-install \
  --name vm1 \
  --memory 2048 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/vm1.qcow2,size=20 \
  --os-variant ubuntu22.04 \
  --cdrom /home/user/ubuntu-22.04.5-desktop-amd64.iso \
  --network network=bridged-network \
  --graphics vnc
```

8.2 Create vm2

```
sudo virt-install \
  --name vm2 \
  --memory 2048 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/vm2.qcow2,size=20 \
  --os-variant ubuntu22.04 \
  --cdrom /home/user/ubuntu-22.04.5-desktop-amd64.iso \
  --network network=bridged-network \
  --graphics vnc
```

8.3 Connect to VM Consoles to Complete Installation

```
virt-viewer --connect qemu:///system --wait vm1
virt-viewer --connect qemu:///system --wait vm2
```

Complete the Ubuntu installation inside each VM console, then let them reboot. Once both VMs are at their desktops/login prompts, proceed.

Step 9 — Get VM IP Addresses

```
# From the host:
sudo virsh net-dhcp-leases bridged-network

# Or check from inside each VM:
ip a show enp1s0    # (interface name may vary inside VM)
```

Step 10 — Test Network Connectivity

Test 1 — VM to VM

```
# On vm1 — ping vm2's IP:
ping 192.168.1.51
# Expected: 0% packet loss
```

Test 2 — VM to Host

```
# On vm1:
ping 192.168.1.50    # br0 IP on host
```

Test 3 — Host to VM

```
# On CentOS host:
ping 192.168.1.51    # vm1 IP
```

Test 4 — Internet from VM

```
# On vm1 or vm2:
ping 8.8.8.8
# Expected: replies from Google DNS
```

Test 5 — LAN Device to VM

```
# From another PC on the same network:
ping 192.168.1.51
# Expected: replies — VM is visible on LAN
```

> ⚠ **WARNING**

> If tests 4 or 5 fail: check firewalld (Step 5) and confirm br0 is in the trusted zone. Also run: sudo sysctl net.ipv4.ip_forward — it must return 1.

Enable IP Forwarding (if needed)

```
# Check current value:
sudo sysctl net.ipv4.ip_forward

# If it returns 0, enable it permanently:
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-ip-forward.conf
sudo sysctl -p /etc/sysctl.d/99-ip-forward.conf
```

Quick Reference — Useful Commands

Command	Purpose
`virsh list --all`	List all VMs and their state
`virsh start vm1`	Start vm1
`virsh shutdown vm1`	Graceful shutdown
`virsh destroy vm1`	Force stop vm1
`sudo brctl show`	Show bridge and attached interfaces
`nmcli connection show`	Show all network connections
`sudo virsh net-list --all`	List libvirt networks
`sudo virsh net-dhcp-leases bridged-network`	Show VM IP addresses
`sudo firewall-cmd --list-all --zone=trusted`	Show trusted zone rules
`sudo getenforce`	Check SELinux mode
`sudo setenforce 0`	Temporarily disable SELinux (testing only)

Troubleshooting

Problem	Fix
br0 has no IP address	Run: sudo nmcli connection down br0 && sudo nmcli connection up br0
VM cannot ping internet	Check firewalld: sudo firewall-cmd --zone=trusted --list-interfaces — br0 must be listed
Permission denied on disk image	Run SELinux restorecon: sudo restorecon -Rv /var/lib/libvirt/images/

virsh net-start fails	Check if br0 is up: ip a show br0 — the bridge must exist before defining the network
virt-install os-variant error	Run: sudo dnf install -y osinfo-db && virt-install --os-variant list | grep ubuntu
LAN cannot reach VM	Ensure VM got a real LAN IP (not 192.168.122.x which is the default NAT network)

Conclusion

By following this guide on CentOS Stream 9 you have:

- Installed KVM, QEMU, libvirt and all required packages via dnf
- Created a Linux bridge (br0) using nmcli / NetworkManager
- Configured firewalld to allow VM network traffic
- Set correct SELinux contexts for libvirt disk images
- Defined a bridged network in libvirt
- Created two guest VMs (vm1 and vm2) attached to that bridge
- Verified full connectivity: VM to VM, VM to Host, VM to Internet, and VM to LAN

The two key differences from Ubuntu are: using nmcli instead of Netplan for bridge configuration, and handling SELinux + firewalld which are enabled by default on CentOS Stream 9.